Reviv-AI-I: Official Submission & Stage-Directed Pitch Dossier

Razorpay AI Buildathon 2026 — Track 3: AI Revenue Recovery

Candidate Name:	Mohammad Zishan Alam	**Email:**	alamzishan07@gmail.com
Selected Track:	Track 03 — AI Revenue Recovery	**GitHub Repo:**	github.com/MohammadZishanAlam/reviv-ai-l
Tech Stack:	FastAPI, SQLite, SQLAlchemy, WebSockets	**AI Engine:**	Gemini 2.0 Flash + Heuristic Failover Core

SECTION 1: THE FOUR CORE EVALUATION ANSWERS (FOR GOOGLE FORM)

1. Problem Taste — Did you pick something that actually matters?

In Indian digital commerce, 15% to 25% of checkouts fail prematurely due to transient bank Core Banking System (CBS) downtime, UPI ceiling limits, or OTP session friction. Today, merchants lose this revenue in one of two broken ways: (1) They do nothing, forfeiting up to 20% of GMV. (2) They use 'dumb dunning'—spamming customers with immediate SMS retry links while the issuer bank is experiencing an active outage. The customer clicks, it fails again, and they abandon the merchant permanently. We built **Reviv-AI-I** because payment failure recovery directly expands GMV for merchants and transaction fee revenue for Razorpay. Rather than creating a generic shopping chatbot, we engineered an autonomous recovery layer addressing the single biggest drop-off point in the payments funnel.

2. Build Quality — Does it run, is it structured, would you trust it?

Reviv-AI-I is a production-grade, full-stack application backed by an automated test suite (9/9 tests passing):
• **Architecture & Persistence:** Powered by a FastAPI backend with SQLite/SQLAlchemy persistence, modeling transactions, recovery attempts, bank health telemetry, and audit trails.
• **Cryptographic Security:** Ingests live Razorpay webhooks with mandatory HMAC-SHA256 signature verification matching Razorpay's exact spec (x-razorpay-signature). Tampered payloads are rejected with HTTP 400.
• **Idempotency Gate:** Webhook deliveries are deduplicated by transaction ID, preventing duplicate dunning dispatches or double charges.
• **Batch Benchmark Engine:** Features a 25-transaction batch benchmark engine (via POST /api/simulate/batch & CLI batch_recovery_test.py) validating the Track 3 bar: measured recovery, compliant escalation, stopping rules, and audit logs.
• **Real Tool Execution:** Interfaces directly with Razorpay's REST APIs (/v1/payment_links) to generate dynamic, bounded recovery payment links with custom expiry TTLs.
• **Real-Time Command Center:** A reactive dashboard connected via WebSockets (/ws) providing live GMV metrics, an interactive 1-click failure simulator, batch benchmark modal, and realistic WhatsApp customer outreach previews.

3. AI Judgment — The right tool in the right place, and where you chose not to use one.

Our design principle was strict: **LLMs for semantic error diagnosis; deterministic code for financial math and execution.**
Where we used AI: (1) Unstructured Error Taxonomy: We use Google Gemini Flash to parse raw bank codes (error_source, error_step, error_reason) and telemetry into 5 operational classes (BANK_DOWNTIME_TRANSIENT, USER_LIMIT_EXCEEDED, AUTH_FRICTION_TIMEOUT, PAYMENT_METHOD_INELIGIBLE, HARD_FAILURE_IRRECOVERABLE). (2) Empathetic Outreach: The agent synthesizes polite, context-aware recovery notifications tailored to the root cause.
Where we explicitly avoided AI: (1) Financial Math: Subtotals, paise-to-rupee conversions, and dynamic 5% cart recovery discounts are 100% deterministic code. (2) Hard Stopping Rules: If a card is reported stolen, blocked, or fraudulent (HARD_FAILURE_IRRECOVERABLE), an LLM cannot negotiate or override. Dunning halts immediately with zero outreach, and an immutable audit log is generated.

4. Failure Recovery — What broke, and what you did about it.

We addressed failure recovery on two fronts:
1. **The Domain Level (Payment Failure Resiliency):** When an issuer CBS experiences an outage (e.g., SBI UPI 504 timeouts), immediate retries cause retry storms. Reviv-AI-I implements an issuer health circuit breaker: if bank health is degraded, outreach is automatically queued with a 15–30 minute delay, re-engaging the buyer only when bank rails stabilize.
2. **The Engineering Level (System Resilience):** (a) *LLM Failover:* External AI APIs can encounter rate limits or timeouts. We engineered a dual-engine architecture: if Gemini fails or times out, the system fails over to a deterministic heuristic rule matrix in <5ms. Recovery is never blocked. (b) *Webhook Race Conditions:* Razorpay retries unacknowledged webhooks. We fixed duplicate links with an atomic database check and event lock before orchestrating actions. (c) *Windows Console Compatibility:* CLI test runners threw UnicodeEncodeError on Windows CP1252 consoles. We added sys.stdout.reconfigure(encoding='utf-8') for clean cross-platform execution.

SECTION 2: COMPLETE STAGE-DIRECTED VIDEO PITCH & DEMO SCRIPT

Screen Setup Instructions: Put your PowerShell terminal on the **Left Half** of your screen and your Browser at http://localhost:8000 on the **Right Half**. This split view proves live backend execution.

Timestamp & What to Open/Click	Exact Directional Speech (Word-for-Word What to Say)
[0:00 – 0:40] **OPEN ON SCREEN:** **GitHub Repository page: github.com/MohammadZishanAlam/reviv-ai-l** **POINT MOUSE TO:** **Project Title & Badges.**	"Hi everyone, I'm Mohammad Zishan Alam, and this is **Reviv-AI-l**—an autonomous revenue recovery engine built for Track 3 of the Razorpay AI Buildathon. In Indian e-commerce, 15 to 25% of checkouts fail. The problem isn't that customers don't want to buy—it's that they hit transient bank server downtime, daily UPI ceiling limits, or OTP friction. Today, merchants either do nothing, losing millions in GMV, or they use *dumb dunning*—spamming customers with immediate retry SMS links while the issuer bank is actively down, causing repeat failures and customer churn. I built Reviv-AI-l to solve this at the infrastructure level: intercepting Razorpay failure webhooks, diagnosing root cause with bank telemetry, and executing bounded recovery workflows that protect merchant reputation."
[0:40 – 1:20] **SWITCH WINDOW TO:** **Split Screen:** **• Left: Terminal with uvicorn server running.** **• Right: Browser at http://localhost:8000.** **POINT MOUSE TO:** **Terminal logs, then to the green '● Agent Active' badge.**	"Notice my screen setup: on the left is our FastAPI backend server, and on the right is the live Merchant Command Center connected via WebSockets. Before testing, let me highlight three critical engineering decisions I made in the backend: 1. **Cryptographic Security:** I implemented strict HMAC-SHA256 signature verification matching Razorpay's exact spec. Tampered payloads are rejected with HTTP 400. 2. **Idempotency Gate:** Webhook delivery is at-least-once. I added atomic deduplication by transaction ID to prevent duplicate dunning links. 3. **Dual-Engine Failover:** External LLMs can time out. While we use Gemini 2.0 Flash for semantic reasoning, if the API experiences latency, the system automatically fails over in under 5 milliseconds to a deterministic heuristic rule engine."
[1:20 – 2:10] **ACTION TO EXECUTE:** **On the right panel, under 'Interactive Failure Simulator', CLICK:** **[Scenario A: SBI CBS Server Downtime]** **POINT MOUSE TO:** **1. Left terminal (shows incoming POST log).** **2. Right card (shows 'Delay: 15m (Bank Down)').**	"Let's test **Scenario A: SBI CBS Downtime**. Watch the terminal on the left as I click. *[Click Scenario A]* Look at the terminal: it immediately ingested the payment.failed webhook and inspected SBI telemetry. Now look at the generated card on the right: Notice the AI's decision: it diagnosed the 504 gateway lag and enforced an **adaptive 15-minute retry delay**. It purposely does NOT dispatch immediately, because retrying right now would guarantee a second failure. It waits until the bank rail stabilizes."

[2:10 – 3:00] **ACTION TO EXECUTE:** **On the simulator panel,** **CLICK:** **[Scenario C: High-Cart OTP Dropoff]** **THEN CLICK:** **[Preview Outreach] button on the new card.** **POINT MOUSE TO:** **1. '+5.0% Incentive' badge.** **2. WhatsApp modal with the dynamic link.**	"Now let's test **Scenario C: High-Cart Friction** with an ■8,999 order. *[Click Scenario C]* The cart value is high, and the failure was OTP session friction. Notice the green badge on the card: the agent dynamically attached an authorized **5% recovery incentive** to close the sale. Let's see what the buyer experiences: I will click **Preview Outreach**. *[Click Preview Outreach]* This opens the exact WhatsApp message the customer receives with their personalized dynamic Razorpay checkout link. Also note our **Stopping Rule**: if a card is stolen or blocked, the agent is hardcoded to abort dunning immediately to protect compliance."
[3:00 – 3:30] **ACTION TO EXECUTE:** **1. Click [Done] to close modal.** **2. On the left panel, click:** **[Run 25-Record Batch Benchmark]** **POINT MOUSE TO:** **1. Batch Audit Modal pops up.** **2. Total Rescued GMV & Recovery Rate.** **3. Stopping Rules: 3 fraud stopped.** **4. Bank Down: 8 delayed 15m.** **5. SQLite Audit Trail verified.**	"Now, let's address the specific bar for Track 3: *Show measured money recovered across a batch, with compliant escalation, stopping rules, and an audit trail.* Let's click **Run 25-Record Batch Benchmark**. *[Click Batch Benchmark button]* Look at the audit breakdown that just ran across 25 diverse transactions: • **Measured Recovery:** Over ■1.3 Lakhs in GMV recovered with a 70%+ recovery rate. • **Stopping Rules:** 3 stolen/compromised cards were detected and halted with zero outreach. • **Compliant Escalation:** 8 transactions on degraded bank rails were delayed 15 minutes to prevent spamming. • **Audit Trail:** All 25 events are cryptographically recorded in our SQLite audit log."
[3:30 – 4:00] **SWITCH WINDOW TO:** **Terminal (Full Screen).** **ACTION TO EXECUTE:** **Run in terminal:** **.\.venv\Scripts\python.exe -m pytest tests/ -v** **POINT MOUSE TO:** **9 passed in <1.0s.**	"To conclude with **Failure Recovery and Build Quality**: during development on Windows, CLI runners faced a UnicodeEncodeError on CP1252 consoles. I resolved this by reconfiguring stdout in the Python runtime. Let's run our automated test suite in the terminal: *[Run pytest command]* All 9 tests pass in under one second—verifying HMAC verification, error classification, dynamic payment links, fraud stopping rules, and database integrity. The full IEEE 830 specification, CLI batch runner, and codebase are live on my GitHub repository. Thank you!"